「送春而血泪满腮，悲秋而红颜惨目」

Wi. Zhimu Yang

Hydrangea Academy of Magic, HAM

Magnolia Institute of Science, MIS.HAM

计算理论导引（3）

> *Type:*
>
> *~~Tutorial~~ / Note*
>
> *For:*
>
> *Amateurs / ~~Professionals~~*
>
> *Preface:*
>
> *我深切感受到时间的缺位和命运的催促，让我无法对那些于我无用的细节抱有同情。*
>
> *Reference:*
>
> *[1] Michael Sipser . 《Introduction to the Theory of Computation》. 机械工业出版社, 2015*

二、Computability Theory(可计算性理论)

1.The Church-Turing Thesis(Church-Turing论题)

Turing Machines

图灵机是更为精确的通用计算模型，能模拟实际计算机的所有行为。图灵机也无法解决的问题则被我们认为在计算上是不可解的。

图灵机与下推自动机的区别在于图灵机将有限制的栈换成了无限长的纸带，纸带上可以存储任意信息，读写头可读可写的操作可以在纸带上正向反向地移动，从而拥有了**无限大的存储容量**和**访问任意数据**的能力，足以模拟实际计算机的所有行为。

Definition of Turing Machines

> *Definition*
>
> *图灵机可描述为一个7元组*$(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$
>
> Q：*State(状态集)，有穷集合，所有状态集合*
>
> Σ：*Input Alphabet(输入字母表)，有穷集合，所有输入允许的字符集合，不包括特殊空白符号*$\sqcup$
>
> Γ：*Stack Alphabet(纸带字母表)，有穷集合，所有纸带允许的字符集合，*
> $\sqcup \in \Gamma, \Sigma \subset \Gamma$
>
> δ：$Q \times \Gamma \to Q \times \Gamma \times \{L, R\})$，*Transition Function(转移函数)*
>
> $\delta(q, a) = (r, b, L/R)$：*位于状态q，读写头对应纸带符号a时，按照转移函数写下符号b以取代a，并向左(L)/右(R)移动，进入状态r*
>
> q_0：*Start State(起始状态)，*$q_0 \in Q$
>
> q_{accept}：*Accept States(接受/终止状态集)，*$q_{accept} \subseteq Q$
>
> q_{reject}： *Reject States(拒绝状态集)，*$q_{reject} \neq q_{accept}$

Computation of Turing Machines

> *Definition*
>
> *图灵机*$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$*，以最左边的n个纸带方格接收输入*$w = w_1 w_2 \cdots w_n \in \Sigma^*$*，其余部分保持为空，读写头自左向右运行，按照转移函数描述规则进行。计算持续到进入接受或拒绝状态后，停机，否则一直运行。*

> *Definition*
>
> *我们以一个Configuration(格局) uqv表示：当前状态为q， u为当前读写头左边的字符串，v为当前读写头以及其右边的字符串*
>
> *如果图灵机能合法地从格局*C_1*一步进入*C_2*，则称格局*C_1*产生格局*C_2*。*

> *Start Configuration：起始格局*
>
> *Accepting Configuration：接受格局*
>
> *Rejecting Configuration：拒绝格局*
>
> *Halting Configuration：停机格局*

> *Definition*
>
> *图灵机M接受输入w，若存在格局的序列$C_1, C_2, \cdots, C_k$，使得：*
>
> 1. *C_1是M在输入w上的起始格局*
> 2. *每一个C_i产生C_{i+1}*
> 3. *C_k是接受格局*
>
> *则称M接受的字符串集合为M的语言，或称语言被M识别。*
>
> *如果一个语言能被某一图灵机识别，则称该语言是Turing-Recognized(图灵可识别的)。*

图灵机有很多的变体，但是它的变体都被证明与图灵机是等价的，如多带图灵机、非确定型图灵机等，非确定性不会影响图灵机的识别能力。图灵机与其他的弱计算模型相比，最大的特征是**能够无限制地访问无限的存储器**，并且能够证明，在一定条件下，**具有此特点的所有计算模型在能力上是等价的**，这可以反应为——在程序设计中，一个算法可以在多种程序设计语言上运行，即两个语言描述了完全相同的算法类。

Decide

图灵机接受一个输入时，可能出现三种结果：接受（停机）、拒绝（停机）、循环（不停机）。我们并不能很好地分辨机器究竟是进入了循环还是运行时间很长，所以我们倾向于设计对于所有输入都停机的图灵机，称为Decider(判定器)（因为对于任意输入总能得到接受或是拒绝），判定器永不循环。如果判定器识别某语言，则称它**Decide(判定)**该语言。

> *Definition*
>
> *如果一个语言可被某一图灵机判定，则称它是Turing Decidable(图灵可判定的)，简称Decidable(可判定的)。*

> *Corollary*
>
> *A language is decidable if and only if some nondeterministic Turing machine decides it.*

> *一个语言是可判定的，当且仅当存在一台非确定型图灵机能够判定它。*

Algorithm

算法是一套实现指定任务的指令集。说到指令集，我们就该想到，计算模型就是读取处理指令集的。作为最为通用的计算模型——图灵机，图灵机解决指令的方法被证明是和算法的精确定义等价的，并且我们只需要一个事实——**图灵机刻画了所有的算法**。

在此基础上，我们就可以在描述算法时，将算法抽象化。算法描述的详细程度有三种：

1. 形式化描述：详细描述图灵机的状态，转移函数等等，最详尽。
2. 实现描述：描述如何操作读写头、纸带如何存储数据等等。
3. 高层次描述：日常语言描述任务，无需提及如何管理读写头和纸带。

2.Decidability(可判定性)

我们大可有长篇大论的契机，但不会是现在，所以这部分只提及一点——并非所有问题在算法上都是可解的，世间恰恰存在着那些无法通过计算解决的问题，在证明这一点时，我们也常常利用Reduction(归约)——将一个问题转化为另一个问题。

Undecidability

> *Corollary*
>
> *Some languages are not Turing-recognizable.*
>
> *存在不能被任何图灵机识别的语言。*

> *Definition*
>
> *一个语言的补是指由不在此语言中的所有串构成的语言。如果一个语言是一个图灵可识别语言的补集，则称它是Co-Turing-Recognizable(补图灵可识别的)。*

> *Theorem*
>
> *A language is decidable iff it is Turing-recognizable and co-Turing-recognizable.*
>
> *一个语言是可判定的，当且仅当它既是图灵可识别的，也是补图灵可识别的。*

3.Reducibility(可归约性)

Computable Functions

图灵机计算函数的方式是：将函数输入纸带上，开始运行，并以停机后的纸带作为函数输出。

> *Definition*
>
> *图灵机M对于每个输入w都停机并且仅有$f(w)$在纸带上，则称函数$f:\Sigma^* \to \Sigma^*$ 为一个可计算函数。*

Mapping Reducibility

> *Definition*
>
> *语言A是映射可归约到语言B的，当存在可计算函数$f:\Sigma^* \to \Sigma^*$ 使得对于每个输入w有：*
>
> $$w \in A \Leftrightarrow f(w) \in B$$
>
> *记作$A \leq_m B$。称函数f为从A到B的归约。*

有了映射归约性，我们就可以利用归约f把问题“$w \in A$?”转换为“$f(w) \in B$?”。

> *Theorem*
>
> *If $A \leq_m B$ and B is decidable,then A is decidable.*
>
> *如果有从A到B的归约f且B是可判定的，则A也是可判定的。*
>
> *Corollary*
>
> *If $A \leq_m B$ and A is undecidable,then B is undecidable.*
>
> *如果有从A到B的归约f且A是不可判定的，则B也是不可判定的。*

> *Theorem*
>
> *If $A \leq_m B$ and B is Turing-recognizable,then A is Turing-recognizable.*

> *如果有从A到B的归约f且B是图灵可识别的，则A也是图灵可识别的。*
>
> *Corollary*
>
> *If $A \leq_m B$ and A is not Turing-recognizable,then B is not Turing-recognizable.*
>
> *如果有从A到B的归约f且A不是图灵可识别的，则B也不是图灵可识别的。*

三、Complexity Theory(复杂性理论)

如果求解一个问题需要过多的时间和空间资源，那么即使理论上可行的问题，实际生活中它仍然是不可解的。

1.Time Complexity(时间复杂性)

对于一个算法，它所使用的步数可能与若干参数有关，我们无法考量在多种情况下多种问题中的参数重要性，所以我们一切从简——把算法的运行时间作为表示输入字符串长度的函数来计算，即运行时间=f(字符串长度)，而不考虑其他参数。

Worst-Case Analysis：在某特定长度的所有输入上的**最长**运行时间。

Average-Case Analysis：在某特定长度的所有输入上的**平均**运行时间。

> *Definition*
>
> *M是一个在所有输入上都停机的确定型图灵机。M的运行时间/时间复杂度是一个函数$f: N \to N$。*
>
> *N：非负整数集合*
>
> *$f(n)$：M在所有长度为n的输入上运行时所经过的最大步数*
>
> *若$f(n)$是M的运行时间，则称M在时间$f(n)$内运行，M是$f(n)$时间图灵机。*

> *Definition*
>
> *N是一个非确定型图灵机且是判定机。N的运行时间/时间复杂度是一个函数$f: N \to N$。*
>
> *N：非负整数集合*
>
> *$f(n)$：N在所有长度为n的输入上所有计算分支/情况中的最大步数*

Asymptotic Analysis

渐近分析与计算多项式的思想相同，我们无法精确求得或者很难求得算法的运行时间。而最高次项对结果的影响起决定性作用，所以我们忽略其他所有项，仅考虑**最高次项**。渐近记法允许我们在机器描述中忽略那些对运行时间的影响不超过常数倍的操作细节。

> *Definition*
>
> *O Notation*
>
> *两个函数f、g有$f: N \to R^+$，$g: N \to R^+$。若存在正整数c和n_0，使得对所有$n \geq n_0$有：*
>
> $$f(n) \leq cg(n)$$
>
> *则称$f(n) = O(g(n))$，直观来说就是函数$f(n)$渐近地不大于函数$g(n)$。*
>
> *当$f(n) = O(g(n))$时，称$g(n)$是$f(n)$的Upper Bound(上界)，或更准确地，$g(n)$是$f(n)$的Asymptotic Upper Bound(渐近上界)。*

我们可以把O看作一个隐藏的常数，即忽略掉一个常数因子的差别时，f将小于等于g。对于$O(n)$时间，我们也通常称为**Linear Time(线性时间)**。

> *Definition*
>
> *o Notation*
>
> *两个函数f、g有$f: N \to R^+$，$g: N \to R^+$。若：*
>
> $$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$
>
> *则称$f(n) = o(g(n))$。直观来说就是函数$f(n)$渐近地小于函数$g(n)$.*
>
> *换言之：*
>
> *对于任意实数$c > 0$，存在n_0，使得对所有$n \geq n_0$有：*
>
> $$f(n) < cg(n)$$

> *Definition*
>
> *Time Complexity Class*

> *对于函数$t: N \to R^+$，时间复杂性类$TIME(t(n))$是由$O(t(n))$时间的图灵机判定的所有语言的集合。*

Complexity Relationships Among Models

在可计算性理论中，丘奇-图灵论题断言，所有合理的计算模型都是等价的，即它们所判定的语言类都是相同的。而在复杂性理论中，模型的选择会影响语言的时间复杂度，同一个语言在不同模型上可能需要不同时间。

> *Theorem*
>
> *函数$t(n) \geq n$，则每一个$t(n)$时间的多带图灵机都和某一个$O(t^2(n))$时间的单带图灵机等价。*

也即，问题的时间复杂度在确定型单带和多带图灵机上最多是平方或**多项式**的差异。

> *Theorem*
>
> *函数$t(n) \geq n$，则每一个$t(n)$时间的非确定型单带图灵机都和某一个$2^{O(t(n))}$时间的确定型单带图灵机等价。*

也即，问题的时间复杂度在确定型和非确定型图灵机上最多是**指数**的差异。

The Class P

运行时间的多项式差异被认为是较小的，而指数差异被认为是较大的。所有合理的确定型计算模型都是Polynomially Equivalent(多项式等价的)。我们在追求“计算”真理的路途中，一步一步地，先忽略常数因子，再忽略比常数因子大得多、却比指数小的多项式，

> *Definition*
>
> *The Class P(Polynomial Time)*
>
> *P是确定型单带图灵机在多项式时间内可判定的语言类，即：*
>
> $$P = \bigcup_k TIME(n^k)$$

- 对于所有与确定型单带图灵机多项式等价的计算模型来说，P是不变的。

P在数学上是一个稳健的类，它不受所采用的具体计算的模型的影响。

- P大致对应于在计算机上实际可解的那一类问题。

当一个问题在P中，那么就意味着存在方法能够在n^k时间内求解它。

在我们实际的应用中，把多项式时间作为实际可解性的标准是有效的，通常的实用就是能否为需要指数时间的问题找到了多项式时间算法。

The Class NP

然鹅很多问题我们暂时没能找到相应的多项式时间算法，它们可能就无法在多项式时间内解决，也可能它们是基于未知原理的多项式时间算法。

其实某些时候，验证解的存在性比确定算法的存在性要更容易，这类问题通常具有**Polynomial Verifiability(多项式可验证性)**。

> *Definition*
>
> *Verifier(验证机)是一个算法V，它对语言A有：*
>
> $$A = \{w | \text{对某个字符串}c，V\text{接受}\langle w, c\rangle\}$$
>
> *验证机的时间仅由w的长度来度量。*
>
> *Polynomial Time Verifier(多项式时间验证机)则在w的长度的多项式时间内运行。*
>
> *若语言A有一个多项式时间验证机，则称它为多项式可验证的。*

验证机利用额外信息c来验证字符串w是语言A的成员，该信息称为A的Certificate(证书)或Proof(证明)。对于多项式验证机，证书具有多项式的长度(w的长度)，这是该验证机在它的时间限内所能访问的全部信息长度。

> *Definition*
>
> *The Class NP(Nondeterministic Polynomial Time)*
>
> *NP是具有多项式时间验证机的语言类。*
>
> *P是NP的子集。*

> *Theorem*

> *A language is in NP iff it is decided by some nondeterministic polynomial time Turing machine.*
>
> *一个语言在NP中，当且仅当它能被某非确定型多项式时间图灵机判定。*

> *Definition*
>
> *Nondeterministic Time Complexity Class*
>
> $NTIME(t(n)) = \{L|L是一个被O(t(n))时间的非确定型图灵机判定的语言\}$

> *Corollary*
>
> $$NP = \bigcup_k NTIME(n^k)$$

NP类对于合理的非确定型计算模型的选择不敏感，因为所有合理的非确定型计算模型都是多项式等价的。NP是在非确定型图灵机山多项式时间内可解的语言类。

P=成员资格可以在多项式时间内**判定**的语言类

NP=成员资格可以在多项式时间内**验证**的语言类

直觉上看，多项式可验证性似乎比多项式可判定性包罗的范围要大，但是现在我们也没法证明NP中存在一个不属于P的语言，也就是不知道“N=NP”是否成立。

NP-Completeness

NP中的一些问题的复杂性与整个NP类的复杂性相关，称这些问题为NP-Complete(NP完备/NP完全)，当任何一个NP完备问题存在多项式时间算法，那么所有NP问题都将是多项式时间可解的。这样一来，想要探究“N=NP”问题的真相，就可以从NP完备问题入手。

Polynomial Time Reducibility

> *Definition*
>
> *若存在多项式时间图灵机M，使得在任何输入w上，M停机时$f(w)$恰好在纸带上，则称函数$f: \Sigma^* \to \Sigma^*$为**Polynomial Time Computation Function(多项式时间可计算函数)**。*

> *Definition*

> *对语言A、语言B，若存在多项式时间可计算函数$f:\Sigma^* \to \Sigma^*$，对于每一个w，有：*
>
> $$w \in A \Leftrightarrow f(w) \in B$$
>
> *则称语言A**多项式时间映射可归约**到语言B，简称多项式可归约，记作$A \leq_P B$，函数f称为A到B的多项式时间归约。*

多项式时间可归约性是映射归约性的有效近似，它把对成员A的资格判定转化为对成员B的资格判定。

> *Theorem*
>
> *若$A \leq_P B$且$B \in P$，则$A \in P$。*

> *Definition*
>
> *NP-Completeness*
>
> *如果语言B满足：*
>
> 1. *B属于NP*
> 2. *NP中的每个A都多项式时间可归约到B。*
>
> *则称语言B是NP完备的。*

> *Theorem*
>
> *If B is NP-complete and $B \in P$, then $P = NP$.*

> *Theorem*
>
> *If B is NP-complete and $B \leq_P C$ for C in NP, then C is NP-complete.*

2.Space Complexity(空间复杂性)

除开时间以外，我们还需要度量算法所消耗的空间，空间的直观表示就是图灵机纸带被使用的格子数。

> *Definition*
>
> ***Space Complexity***

> *M是一个在所有输入上都停机的确定型图灵机，M的空间复杂度是一个函数* $f: N \to N$*。*
>
> $f(n)$*：M在任何长为n的输入上扫描纸带方格的最大数。*
>
> *若M空间复杂度为* $f(n)$*，则称M在空间* $f(n)$ *运行。*
>
> *如果M是对所有输入在所有分支上都停机的非确定型图灵机，则它的空间复杂度* $f(n)$ *定义为M对任何长为n的输入，在任何计算分支上所扫描的纸带方格的最大数。*

> *Definition*
>
> *对于函数* $f: N \to R^+$*，Space Complexity Class(空间复杂性类)* $SPACE(f(n))$ *和* $NSPACE(f(n))$ *有如下定义：*
>
> $$SPACE(f(n)) = \{L | L\text{是被}O(f(n))\text{空间的确定型图灵机判定的语言}\}$$
> $$NSPACE(f(n)) = \{L | L\text{是被}O(f(n))\text{空间的非确定型图灵机判定的语言}\}$$

Savitch's Theorem

Savitch定理表明，确定型机器可以用非常少的空间模拟非确定型机器。在时间复杂性中，实现这样的模拟需要指数倍地增加时间；在空间复杂性中，该定理说明，任何消耗 $f(n)$ 空间的非确定型图灵机都可以转变为仅消耗 $f^2(n)$ 空间的确定型图灵机。

> *Theorem*
>
> *Savitch's Theorem*
>
> *For any function* $f: N \to R^+$*, where* $f(n) \geq n$*,*
> $NSPACE(f(n)) \subseteq SPACE(f^2(n))$

The Class PSPACE

与定义P类相似，我们为空间复杂性定义PSPACE类。

> *Definition*
>
> *PSPACE是在确定型图灵机上、多项式空间内可判定的语言类，即：*

> $$PSPACE = \bigcup_k SPACE(n^k)$$

我们当然可以类似地定义非确定型图灵机对应的NSPACE类，但是根据Savitch定理所说，其实NPSPACE=PSPACE。

Relationships Between Space and Time Complexity

$$P \subseteq PSPACE$$
$$NP \subseteq NPSPACE \Rightarrow NP \subseteq PSPACE$$

当时间$t(n) \geq n$时，在每个计算步上最多能访问一个新单元，因此，任何在时间$t(n)$内运行的机器最多能消耗$t(n)$空间。 直观来说就是，运行快的机器不可能消耗大量空间。

同理，消耗$f(n)$空间的图灵机必定在时间$f(n)2^{O(f(n))}$内运行，$PSPACE \subseteq EXPTIME = \bigcup_k TIME(2^{n^k})$，我们可以得到复杂性类之间的关系：

$$P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME$$

PSPACE-Completeness

同样的我们引入PSPACE完备类。

> *Definition*
>
> *若语言B满足：*
>
> 1. *B属于PSPACE*
> 2. *PSPACE中的每一个语言A可多项式时间归约到B*
>
> *则称它是PSPACE完备的。*

我们可以这样理解完备问题——完备问题是一个类中最难的问题，如果有办法求解完备问题，那么类中的其他问题也就迎刃而解了。

The Class L and NL

目前为止的讨论中，我们都把复杂性界限视为至少是线性的情况，即$f(n)$至少是n。接下来考虑更小的Sublinear(亚线性)的情况。在时间复杂性中，亚线性界限甚至不够读完输入，故不考虑。在空间复杂性中，亚线性界限可以读完输入但没有空间存储输入，所以我们使用一些变种计算模型——具有一条只读工作带和一条读写工作带的双带图灵机，只有工作带上被扫描的单元构成这种形式图灵机的空间复杂性。

> *Definition*
>
> *L是确定型图灵机在**对数**空间内可判定的语言类，即：*
>
> $$L = SPACE(\log n)$$
>
> *NL是非确定型图灵机在**对数**空间内可判定的语言类，即：*
>
> $$NL = NSPACE(\log n)$$

NL-Completeness

NL类的问题都在多项式时间内可解，所以我们不能再使用多项式时间可归约性来定义NL完备类，我们只能使用新的对数空间可归约性来解决。

> *Definition*
>
> *Log space transducer(对数空间转换器)是有一条只读输入带、一条只写输出带和一条读写工作带的图灵机。输出带的头无法向左移动读取已写的内容。工作带可以包含$O(\log(n))$个符号。对数空间转换器M计算一个函数$f: \Sigma^* \to \Sigma^*$，$f(w)$是把w放在M的输入带上启动M运行至停机时输出带上存放的字符。称f为对数空间可计算函数。*

> *Definition*
>
> *如果语言A通过对数空间可计算函数f映射可归约到语言B，则称A对数空间可归约到B，记作$A \leq_L B$*

> *Definition*
>
> *如果语言B满足：*
>
> 1. $B \in NL$
> 2. *NL中的每个A对数空间可归约到B*

> *则称B是NL完备的。*

> *Theorem*
>
> $A \leq_L B$*且*$B \in L$*，则*$A \in L$

> *Corollary*
>
> *若有一个NL完全语言属于L，则L=NL。*
>
> $NL \subseteq P$

3.Intractability(难解性)

当计算问题理论上可解却需要大量时间空间资源难以应用时，我们称它为难解的。

Hierarchy Theorems

直觉上来说，我们认为给机器更多的时间和空间它就能解决更多问题，Hierarchy Theorems(层次定理)说明了这种直觉在一定条件下的正确性。

> *Definition*
>
> *函数*$f: N \to N$*，其中*$f(n)$*至少为*$O(\log n)$*，如果函数*f*把n个1组成的字符串映射为*$f(n)$*的二进制表示，并且该函数在空间*$O(f(n))$*内是可计算的，则称该函数为space constructible(空间可构造的)。*

> *Theorem*
>
> *Space Hierarchy Theorem*
>
> *For any space constructible function* $f: N \to N$*，a language A exists that is decidable in* $O(f(n))$ *space but not in* $o(f(n))$ *space.*
>
> *对于任何空间可构造函数*$f: N \to N$*，存在语言A，在空间*$O(f(n))$*内可判定，但不能在空间*$o(f(n))$*内判定。*

任何空间可构造的空间界限的渐近增加都将扩大可判定的语言类。

> *Definition*

> *函数$t: N \to N$，其中$t(n)$至少为$O(n \log n)$，如果函数t把n个1组成的字符串映射为$t(n)$的二进制表示，并且该函数在时间$O(t(n))$内是可计算的，则称该函数为time constructible(时间可构造的)。*

> *Theorem*
>
> *Time Hierarchy Theorem*
>
> *For any time constructible function $t: N \to N$，a language A exist that is decidable in $O(t(n))$ time but not decidable in time $o(\frac{t(n)}{\log t(n)})$.*
>
> *对于任何时间可构造函数$t: N \to N$，存在语言A，在时间$O(t(n))$内可判定，但不能在时间$o(\frac{t(n)}{\log t(n)})$内判定。*

可能对计算理论的导引就到这里了。前些日子看到研究随机性计算理论的科学家荣获图灵奖，同时讶异他竟是数学计算机双料得主，这恰恰坚定了我对数学物理的信心——没有坚实的数物基础，就不可能在计算机领域里结出硕果。一阵澎湃的心潮驱使着我尽快完成计算理论导引的学习并踏足未至之境，可我始终无法摆脱耳边环绕的那些屈服于命运与奴役的呵斥——我们踏破铁鞋觅的是个什么物！我兴许是永远无法原谅这种扼住咽喉奔劳的生活，无法宽恕广工、海大给予我的片刻自由而又姗姗收走，无法原谅我。我做不了，也无法接受，我宁让着我的血肉融进海大东坡湖里，也不要这具枯骨就在这里漫无目的地爬行！